Decentralized Federated Memory for Autonomous Agents

A Local-First, Graph-Based Architecture for Persistent Agent Memory with Cross-Store Traversal

Hung Nguyen, 2026

Abstract

Current approaches to AI agent memory rely on centralized vector databases (Pinecone, Chroma) or hosted services (Mem0, Zep), creating cloud dependencies and single points of failure for what should be a fundamental agent capability. We present **memory-cli**, a local-first, federated, graph-based memory system where autonomous stores discover each other through edge references and cache lookups in their own metadata. The system implements a cognitively-inspired dual-store architecture modeled on Atkinson-Shiffrin memory theory, with a novel hybrid retrieval pipeline combining BM25 full-text search, vector similarity (KNN), Reciprocal Rank Fusion, spreading activation over a directed weighted graph, and exponential temporal decay. Store federation is achieved through scoped neuron handles (`LOCAL-42`, `GLOBAL-42`, `a3f2:42`) and self-describing databases that act as both memory graphs and phonebooks of every store they have communicated with. No central server, registry, or coordination protocol is required.

---

1. Introduction

AI agents forget between sessions. Despite advances in context windows and retrieval-augmented generation, the fundamental problem persists: an agent's knowledge dies with its context. The solutions available today fall into two categories:

1. **Centralized vector stores** (Pinecone, Chroma, Weaviate) — fast retrieval but no graph structure, no association, no consolidation model. Every memory is an isolated point in embedding space.
2. **Hosted memory services** (Memo, Zep, Letta/MemGPT) — richer abstractions but cloud-dependent, with opaque storage and no inter-agent federation.

Neither approach gives agents what humans have: a **local, portable, associative memory** that strengthens through connection and decays through disuse — one that can be shared selectively without centralization.

We present memory-cli, a system built on three insights:

- **Memory is a graph, not a list.** Isolated facts are dots. Connected facts are knowledge. Edges are what make search work.
- **Memory consolidation follows cognitive science.** Quick capture (short-term) and structured filing (long-term) are distinct operations with different UX, mirroring the Atkinson-Shiffrin multi-store model.
- **Federation emerges from self-describing stores.** Each database carries its own identity. Cross-store references are explicit in the handle syntax. Discovery is lazy. No coordinator needed.

---

2. Architecture

2.1 Data Model

The core data model consists of four primitives:

- **Neurons** — content nodes representing memories, facts, or entities. Each neuron carries content, timestamps, project scope, source provenance, and an embedding vector.
- **Edges** — directed, weighted connections between neurons. Each edge has a reason (semantic label) and weight (strength). Edges enable spreading activation traversal.
- **Tags** — categorical labels with AND/OR filtering. Auto-generated tags include date stamps and project identifiers.

- **Attributes** — key-value metadata on neurons for structured properties (type, source, domain-specific fields).

The storage layer uses SQLite with two extensions: **FTS5** for BM25 full-text search and **sqlite-vec** for KNN vector similarity. This combination keeps the entire system in a single portable file with no external dependencies.

2.2 Noun-Verb CLI Grammar

The system exposes a noun-verb command grammar designed for programmatic agent use:

```
memory <noun> <verb> [args] [flags]
```

Six nouns cover the complete operation space: `neuron` (CRUD + search), `edge` (graph connections), `tag` (categorization), `attr` (metadata), `batch` (bulk operations), and `meta` (system introspection). All output is structured JSON by default, making it parseable by agents without string manipulation.

2.3 Storage Backend

Each memory store is a single SQLite database file containing nine tables:

Table	Purpose
`neurons`	Content nodes with timestamps, project, source, status
`edges`	Directed weighted connections between neurons
`tags` / `neuron_tags`	Tag registry and neuron-tag junction
`attr_keys` / `neuron_attrs`	Attribute registry and neuron-attribute junction
`neurons_fts`	FTS5 virtual table for BM25 retrieval
`neurons_vec`	vec0 virtual table for KNN vector retrieval (768-dim)
`meta`	Key-value metadata: schema version, fingerprint, store registry

Triggers maintain FTS5 synchronization automatically on neuron and tag mutations. The vec0 table follows a mandatory two-step query pattern (query standalone, then hydrate) due to sqlite-vec constraints on JOINs.

---

3. Cognitive Architecture: Atkinson-Shiffrin for Agents

3.1 The Multi-Store Model

The Atkinson-Shiffrin model (1968) describes human memory as a sequential process through three stores: sensory memory, short-term memory (STM), and long-term memory (LTM). Information moves through these stores via attention, rehearsal, and elaborative encoding. Critically, **consolidation from STM to LTM requires active processing** — associating new information with existing knowledge.

We map this directly to agent memory operations:

Cognitive Stage	Agent Operation	CLI Command
Sensory input	Conversation context	(in-context, ephemeral)
Short-term capture	Quick, unstructured fact storage	`memory neuron add "..."`
Long-term consolidation	Structured graph with associations	`memory batch load --inline '<yaml>'`

3.2 The Inbox and the Filing Cabinet

`neuron add` is the **inbox** — fast, unstructured, one fact at a time, no edges. It's the equivalent of jotting a note on a sticky pad. The captured neuron exists as an isolated dot in the graph.

`batch load` is the **filing cabinet** — structured YAML documents with multiple neurons, typed edges between them, tags, and attributes. Loading a graph document creates a connected subgraph in a single atomic operation.

```
neurons:
  - ref: interview
    content: "Video interview Friday at 1:00 PM ET"
    tags: [interview, 2026-03-13]
    type: event
  - ref: payam
    content: "Payam Fard — Director of Software Engineering"
    tags: [interview, contact]
    type: person

edges:
  - from: interview
    to: payam
    type: has_interviewer
    weight: 1.0
```

3.3 Why Consolidation Matters for Search

The difference between inbox and filing cabinet is not merely organizational — it is **functional**. Isolated neurons with no edges cannot participate in spreading activation. They are reachable only by direct keyword or vector match. Connected neurons, by contrast, amplify each other: searching for "interview" activates the interview node, which spreads activation to the interviewer, the talking points, and the preparation notes.

Over time, this creates a natural selection pressure: **connected memories persist in search relevance; isolated memories decay**. This mirrors the cognitive science finding that elaborative rehearsal (connecting new information to existing knowledge) is essential for long-term retention.

3.4 Typical Agent Workflow

1. **During work:** `neuron add` for quick observations (short-term capture)
2. **End of session:** `batch load` a graph document that structures what was learned (long-term consolidation)
3. **Over time:** Isolated neurons with no edges decay in search ranking; connected ones persist

---

4. Hybrid Retrieval Pipeline

4.1 Ten-Stage Light Search

The search pipeline implements a ten-stage retrieval process that combines lexical matching, semantic similarity, graph traversal, and temporal awareness:

```
1. Query Embedding (nomic-embed-text-v1.5, asymmetric prefixes)
       ↓
2. BM25 Retrieval (SQLite FTS5, porter stemming)
       ↓
3. Vector Retrieval (KNN via sqlite-vec, cosine distance, 768-dim)
       ↓
4. RRF Fusion (k=60, rank-based, no score calibration needed)
       ↓
5. Spreading Activation (BFS, linear decay, bidirectional edges)
       ↓
6. Temporal Decay (exponential, half-life 30 days)
       ↓
7. Tag Filtering (post-activation AND/OR filter)
       ↓
8. Final Score (multiplicative: retrieval_score × temporal_weight)
       ↓
9. Pagination (--limit / --offset)
       ↓
10. Hydration & Output (neuron fields + score breakdown)
```

4.2 Reciprocal Rank Fusion

BM25 and vector retrieval produce scores on incomparable scales. Rather than attempting score normalization, we use Reciprocal Rank Fusion (Cormack et al., 2009) with k=60:

```
rrf_score(d) = Σ 1/(k + rank_i(d) + 1)
```

where `rank_i(d)` is the 0-based rank of document `d` in retrieval list `i`. Documents appearing in both BM25 and vector results receive additive scores, naturally boosting candidates with both lexical and semantic relevance. RRF is parameter-light and requires no training data or score calibration.

4.3 Spreading Activation

After RRF fusion identifies seed neurons, spreading activation propagates signal through the graph using BFS traversal:

- **Seeds** start with activation = 1.0
- **Linear decay** per hop: `activation = max(0, 1 − (depth + 1) × decay_rate)`
- **Edge weight modulation**: `propagated = activation × edge_weight`
- **Bidirectional traversal**: follows both outgoing and incoming edges
- **Max-score update**: if a neuron is reached via multiple paths, the highest activation wins

With default parameters (decay_rate=0.3, fan_out_depth=1): - Depth 0 (seeds): 1.0 - Depth 1: 0.7 × edge_weight - Depth 2: 0.4 × edge_weight - Depth 3: 0.1 × edge_weight (effective cutoff)

This is computationally lightweight — for S seed neurons at depth 1, the algorithm issues S+1 queries. Even at depth 3, the maximum is 3 query round-trips. The graph structure, not brute-force embedding search, determines what is surfaced.

4.4 Temporal Decay

An exponential decay function biases search toward recent memories:

```
weight(age) = e^(−λ × age_days)    where λ = ln(2) / half_life
```

With the default half-life of 30 days: - Age 0: weight = 1.0 - Age 30 days: weight = 0.5 - Age 90 days: weight = 0.125

Temporal decay is **multiplicative**, not additive — it preserves relative ranking from earlier stages while biasing toward recency. This prevents stale memories from outranking fresh, relevant results while allowing highly-connected older memories to remain discoverable through their activation scores.

4.5 Heavy Search: LLM-Augmented Retrieval

An optional heavy search mode enhances the pipeline with Haiku-powered query expansion and re-ranking:

1. **Inflated retrieval**: Run light search with 3× the requested limit
2. **Re-ranking**: Haiku reorders candidates by query relevance (temperature 0, deterministic)
3. **Query expansion**: Haiku generates 3-8 related search terms (temperature 0.5)
4. **Expansion retrieval**: Each expanded term runs a fresh light search
5. **Merge**: Re-ranked results + expansion results, deduplicated, paginated

Heavy search is transparent — the output schema is identical to light search. Callers can swap `--heavy` in or out without parsing changes. Failures gracefully degrade to light search results.

---

5. Federated Memory Architecture

5.1 Three Scopes

Memory-cli supports three scopes of memory, connected by edges into a single searchable graph:

```
        GLOBAL (~/.memory/)
           /          \
          /            \
    LOCAL-A             LOCAL-B
   (.memory/)          (.memory/)
    project A           project B
```

- **Global** (`~/.memory/`) — user preferences, cross-project knowledge, personal facts
- **Local** (`<project>/.memory/`) — project-specific knowledge, scoped by working directory
- **Foreign** — another project's local store, referenced by database fingerprint

5.2 Scoped Neuron Handles

Every neuron ID returned by the CLI is scoped with a prefix that identifies its origin store:

Handle	Store	Resolution
`LOCAL-42`	Project `.memory/memory.db`	Current working directory
`GLOBAL-42`	`~/.memory/memory.db`	User home directory
`a3f2:42`	Foreign store	Fingerprint lookup

Handles are stored compact (`L-42` , `G-42`) but displayed explicit (`LOCAL-42` , `GLOBAL-42`). Both forms are accepted on input. Bare integers default to the current store context.

This prevents the primary failure mode of bare integer IDs: an agent captures neuron 42 in project A, switches context to project B, and accidentally creates an edge to project B's neuron 42 — a completely different memory.

5.3 Store Fingerprints

Each memory store receives a UUID fingerprint at initialization, written to the `meta` table:

```
INSERT INTO meta (key, value) VALUES ('fingerprint', 'a3f2b7c1');
INSERT INTO meta (key, value) VALUES ('project', 'memory-cli');
INSERT INTO meta (key, value) VALUES ('db_path', '/path/to/.memory/memory.db');
```

The fingerprint is a content-addressable pointer — when an agent needs to reference a neuron from another project's store, it uses `fingerprint:id` syntax. Resolution follows a simple protocol:

1. Check the current store's `meta` table for a cached `store:<fingerprint>` → path mapping
2. If not cached, scan known `.memory/` directories and read their `meta` tables
3. Cache the discovered mapping in the requesting store's `meta` table

Every store is both a memory graph and a phonebook of every other store it has ever communicated with. No external registry file is needed. The database is self-describing.

5.4 Cross-Store Edges

Edges can span store boundaries using scoped handles:

```
neurons:
  - ref: deploy-fact
    content: "project B deploys to staging first"

edges:
  - from: deploy-fact
    to: GLOBAL-42              # user preference: "prefers cautious deploys"
    type: informed_by
  - from: deploy-fact
    to: a3f2:42                # project A: "deploy uses rsync to prod-west-2"
    type: learned_from
```

5.5 Cross-Store Search

Spreading activation does not respect store boundaries — edges are edges regardless of where the target neuron lives. When activation propagates to a foreign handle, the CLI:

1. Resolves the fingerprint to a database path
2. Opens a read-only connection to the foreign store
3. Retrieves the neuron and its local edges
4. Continues propagation if depth allows

This means an agent in project B searching for "deploy process" can discover: - `LOCAL-B:12` — "project B deploys to staging first" (local hit) - `GLOBAL-7` — "user prefers cautious deploys" (global, via edge from LOCAL-B:12) - `a3f2:42` — "project A's deploy uses rsync to prod-west-2" (foreign, via edge from GLOBAL-7)

Three stores, one search, full context. The graph doesn't care where the neurons live.

5.6 Comparison: Git as Precedent

This architecture mirrors Git's decentralized model:

Git	memory-cli
Repository	Memory store
Commit hash	Store fingerprint
Remote	Foreign store reference

Git	memory-cli
`git remote add`	First cross-store edge (auto-discovers and caches)
No central server required	No central registry required
Repos reference each other by hash	Stores reference each other by fingerprint

Git proved that decentralized, content-addressable references could replace centralized version control. We argue the same principle applies to agent memory.

---

6. Related Work

6.1 Centralized Vector Stores

Pinecone, **Chroma**, and **Weaviate** provide fast vector similarity search but operate as isolated embedding stores. They lack graph structure (no edges, no spreading activation), consolidation models (no distinction between short-term and long-term), and federation (single-instance, often cloud-hosted). Memory is reduced to points in embedding space with no associative connections.

6.2 Hosted Memory Services

Memo provides a dedicated memory layer with versioned APIs, MMR-based reranking, and recently added graph memory. However, it operates as a managed SaaS with centralized storage.

Zep stores memory as a temporal knowledge graph tracking fact evolution, combining graph-based memory with vector search. It offers on-premise deployment but requires a managed graph database infrastructure.

Letta (formerly MemGPT) introduced memory management for agents through a virtual memory hierarchy inspired by operating systems, where agents actively manage what stays in-context versus archival storage. This is the closest philosophical ancestor to our approach, but Letta's architecture is agent-runtime-coupled rather than tool-level.

All three approaches require cloud infrastructure or complex self-hosted deployments. None support cross-instance federation or store-level identity for inter-agent memory sharing.

6.3 Spreading Activation in IR

Spreading activation has a rich history in information retrieval and cognitive science. Collins and Loftus (1975) proposed the original spreading activation theory of semantic processing. Recent work by SA-RAG (2025) integrates spreading activation into knowledge-graph-based RAG systems, demonstrating improved document retrieval for complex reasoning tasks. Our implementation differs in combining spreading activation with hybrid retrieval (BM25 + KNN + RRF) and temporal decay in a single pipeline, optimized for agent memory rather than document retrieval.

6.4 Federated Knowledge Graphs

The concept of federated knowledge graphs — distributed multi-source knowledge graphs stored across multiple clients — has been explored primarily in enterprise and privacy-preserving contexts. Existing approaches focus on federated learning over knowledge graph embeddings or federated SPARQL querying. Our approach differs fundamentally: rather than training shared models across distributed graphs, we enable **lazy, edge-driven discovery** where stores learn about each other through explicit cross-store references. No coordination protocol, no shared training, no central schema.

---

7. Design Principles

7.1 Local-First

The entire system runs locally. No cloud dependency, no API keys required (except for optional heavy search). A memory store is a single SQLite file that can be copied, backed up, or moved with standard filesystem operations. This is not a philosophical preference — it is a **reliability requirement** for autonomous agents that must function without network access.

7.2 Opaque Storage

Agents interact with memory exclusively through the CLI. They cannot browse the database, run arbitrary SQL, or inspect the schema. This is deliberate: the storage format can evolve (migrations, schema changes, index optimizations) without breaking agent workflows. The CLI is the API.

7.3 Idempotent Operations

Loading the same graph document twice reuses existing neurons (matched by source + content hash) rather than creating duplicates. This makes batch operations safe to retry and enables incremental graph building across sessions.

7.4 Graceful Degradation

Missing embedding model? Search falls back to BM25-only. Haiku API unavailable? Heavy search degrades to light search. Foreign store unreachable? Cross-store edge is noted but doesn't block the query. Every component is designed to degrade rather than fail.

---

8. Conclusion

Memory-cli demonstrates that agent memory does not require centralized infrastructure. By combining cognitive science (Atkinson-Shiffrin consolidation), information retrieval (hybrid BM25 + vector + spreading activation), and distributed systems (content-addressable federation), we achieve a system where:

- **Memory strengthens through connection**, not just storage
- **Consolidation is an explicit agent action**, not a background process
- **Federation emerges from use**, not configuration
- **Search traverses association**, not just similarity

The three-scope model (LOCAL, GLOBAL, foreign) with scoped handles and self-describing databases provides a foundation for **inter-agent memory sharing** without the coordination overhead of centralized systems. Each store is sovereign, portable, and

self-contained — yet capable of participating in a larger memory graph through explicit edge references.

We believe this architecture points toward a future where AI agents maintain rich, persistent, federated memory graphs that survive across sessions, projects, and teams — without surrendering control to a central service.

References

1. Atkinson, R. C., & Shiffrin, R. M. (1968). Human memory: A proposed system and its control processes. *Psychology of Learning and Motivation*, 2, 89-195.
2. Collins, A. M., & Loftus, E. F. (1975). A spreading-activation theory of semantic processing. *Psychological Review*, 82(6), 407-428.
3. Cormack, G. V., Clarke, C. L., & Buettcher, S. (2009). Reciprocal rank fusion outperforms condorcet and individual rank learning methods. *Proceedings of SIGIR '09*, 758-759.
4. Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S., Stoica, I., & Gonzalez, J. (2023). MemGPT: Towards LLMs as operating systems. *arXiv:2310.08560*.
5. Choudhary, T., et al. (2025). Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv:2504.19413*.
6. Besta, M., et al. (2025). Leveraging spreading activation for improved document retrieval in knowledge-graph-based RAG systems. *arXiv:2512.15922*.
7. Nomic AI. (2024). nomic-embed-text-v1.5: A reproducible long context text embedder. Technical report.
8. SQLite. sqlite-vec: A vector search SQLite extension. https://github.com/asg017/sqlite-vec

---

memory-cli is open source under the MIT license at https://github.com/ai-janitor/memory-cli